MagicCode Super-Resolution Software Integration Guide

Document Version: v2.0.0. Release Date: September 2, 2026.

1. Libraries

Platform	Enable archive	Core archive
Android	lib/android/libmagic_sr_enable.a	lib/android/libmagic_sr.a
iOS	lib/ios/libmagic_sr_enable.a	lib/ios/libmagic_sr.a
macOS arm	lib/mac_arm/libmagic_sr_enable.a	lib/mac_arm/libmagic_sr.a
Windows	lib/windows/libmagic_enable_sr.lib	lib/windows/libmagic_sr.lib

Pick one path. Core-only archives do not export MC_Enable*. Windows temporal objects are inside libmagic_sr.lib; do not also link magic_sr_win_vk / magic_vulkan_temporal / magic_vulkan_taau.

2. Spatial Enable integration

1. Include mc_enable.h and link the Enable archive.
2. MC_Enable_SetModelPath or SetModelDir before the first Enable.
3. On Android GLES/Vulkan, MC_Enable_SetInputSizeHint(width, height).
4. Call MC_Enable(input, scale) each frame; release only with MC_Disable.

3. Temporal session integration

5. Include mc_interface.h and link libmagic_sr.
6. Fill input_param_t (struct_size, temporal alg_mode, scaler in (1, 8], gpu_context, create-stage depth/HDR).
7. MC_Init. Vulkan/D3D11 device import is required as documented in mc_interface.h.
8. MC_Init once and keep the handle across frames. Each frame: zero-init temporal_frame_t, set struct_size, bind depth/motion (and optional masks), set jitter/frame_index/camera, attach to magic_frame_t.frame, bind image_in/image_out (including Vulkan layouts on color/depth/motion/output), set command_buffer on Vulkan.
9. MC_Process only records or dispatches for that frame. On Vulkan it does not submit or wait. Set reset_history on load/teleport.
10. After the caller has submitted and waited on recorded GPU work, MC_Uninit in shutdown. Do not Uninit immediately after MC_Process while GPU work may still be in flight.

Temporal Super-Resolution

v2.0.0 adds temporal super-resolution through the session API: libmagic_sr.a (Windows: libmagic_sr.lib) + mc_interface.h + MC_Init / MC_Process / MC_Uninit. MC_Enable remains a single-texture spatial helper. It may accept a temporal alg_mode_e value by range check, but it cannot supply per-frame depth, motion, or jitter and must not be documented as a complete temporal entry.

Modes

Value	Enumerator	Role
0	SPATIAL_SPEED_MODE	Spatial, throughput first
1	SPATIAL_BALANCED_MODE	Spatial, quality/cost trade-off
2	TEMPORAL_SPEED_MODE	Temporal, throughput first
3	TEMPORAL_BALANCED_MODE	Temporal, canonical FSR-family path

- Temporal scaler_factor is (1, 8]; 1.0 itself is rejected. Spatial remains [1.0, 8.0].
- TEMPORAL_SPEED_MODE on Metal, Vulkan, and OpenGLES currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family temporal path (same family as Balanced, not a separate customer API).
- TEMPORAL_BALANCED_MODE uses the canonical FSR-family path on every GPU temporal backend. Desktop Balanced edge-resolve is supported on Windows Vulkan, Windows Direct3D 11, desktop OpenGL 4.3, and Apple Metal. Android Vulkan and Android OpenGLES do not run edge-resolve. If enable_sharpening is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run temporal modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- magic_frame_t.image_in / image_out: caller-owned color and output GPU resources.
- magic_frame_t.frame: non-NULL temporal_frame_t* with struct_size = sizeof(temporal_frame_t).
- temporal_frame_t.depth and motion at input resolution.
- jitter_offset_x/y in input pixels, top-left, +X right, +Y down.
- frame_index / reset_history; camera_near / camera_far / camera_fov_y (radians).
- gpu_context at MC_Init: Vulkan requires physical_device + device; D3D11 requires device; Metal device may be NULL; GL/GLES use the current context (library never makes current).
- Vulkan temporal requires magic_frame_t.command_buffer to be a recording VkCommandBuffer. Metal command_buffer is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage depth_reversed / depth_infinite / hdr_color on input_param_t (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.

- motion_vector_scale_x/y is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both temporal modes enable depth+MV history rejection by default.
- Diagnostic only: MAGICSR_TAAU_AUTO_MASK=0 and MAGICSR_TAAU_HIST_REJECT=0 can turn those defaults off. They are not a customer configuration path.

4. Unity / Unreal

Plugins wrap Enable for spatial upscaling. They do not provide per-frame depth/motion/jitter. Full temporal belongs on the Native session API. See plugin/unity/USAGE_GUIDE.md and plugin/ue/USAGE_GUIDE.md.

5. Vulkan command-buffer notes

- command_buffer must already be recording. MagicSR does not begin, end, submit, or wait.
- layout on color, depth, motion, and image_out must be the actual layout at entry (not 0 / UNDEFINED).
- Keep the MC handle across frames. Process only records. Caller submit/wait, then MC_Uninit at shutdown.
- physical_device, device, and images must be the same VkDevice.